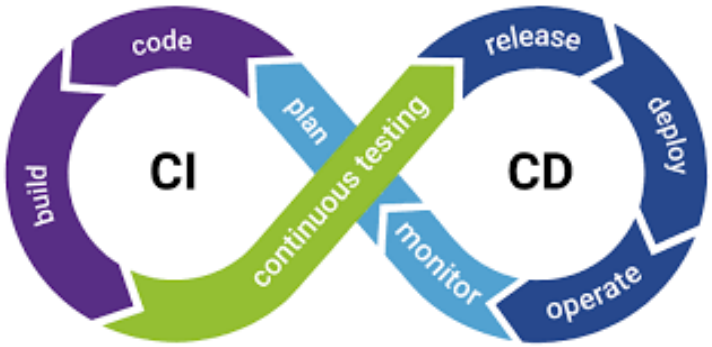
Docker: Tecnología de Contenerización y Orquestación

Esta tecnología creada en 2013 se ha convertido en algo **indispensable** a día de hoy para un administrador de sistemas. También para casi cualquier ámbito relacionado con la informática, especialistas IT o especialista TIC.

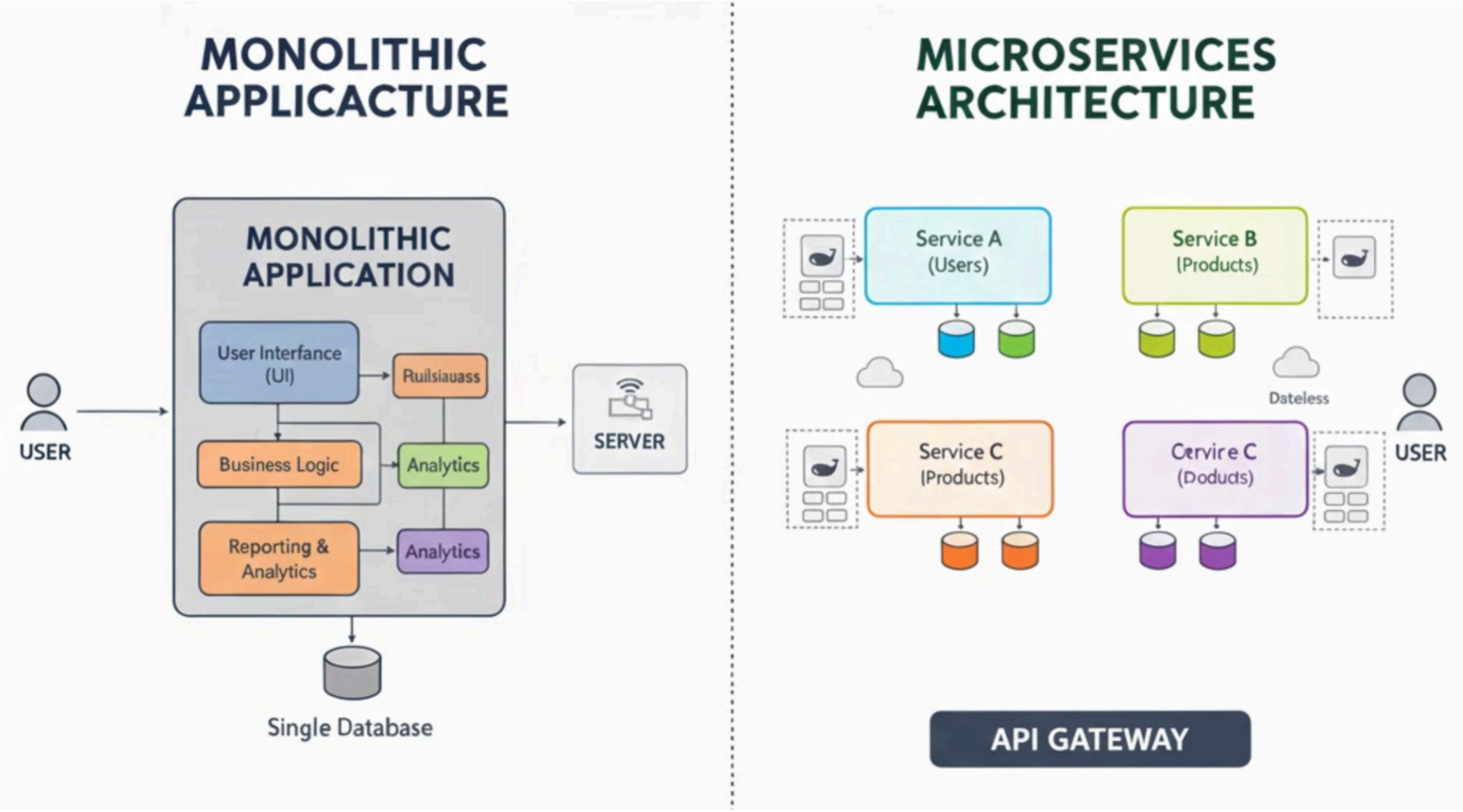
Es de código abierto (Open Source). Y se basa en la **virtualización ligera**, ya que **no** se necesita un **sistema operativo completo**. Solo **una aplicación** en concreto. Como por ejemplo MySQL o Apache.

Es una extensión de los LXC (Contenedores Linux) .



CI/ CD **. integración continua/entrega o implementación continuas**, es una práctica de desarrollo de software que es posible gracias a la automatización. Las actualizaciones frecuentes y confiables aceleran los ciclos de las versiones debido a la entrega continua de código.

Docker supone una revolución en los entornos de CI/CD . Tras la actualización del repositorio de proyecto, se crean contenedores para pasar las pruebas, se construyen las nuevas imágenes y se despliega la nueva versión de la aplicación sin parada del sistema.

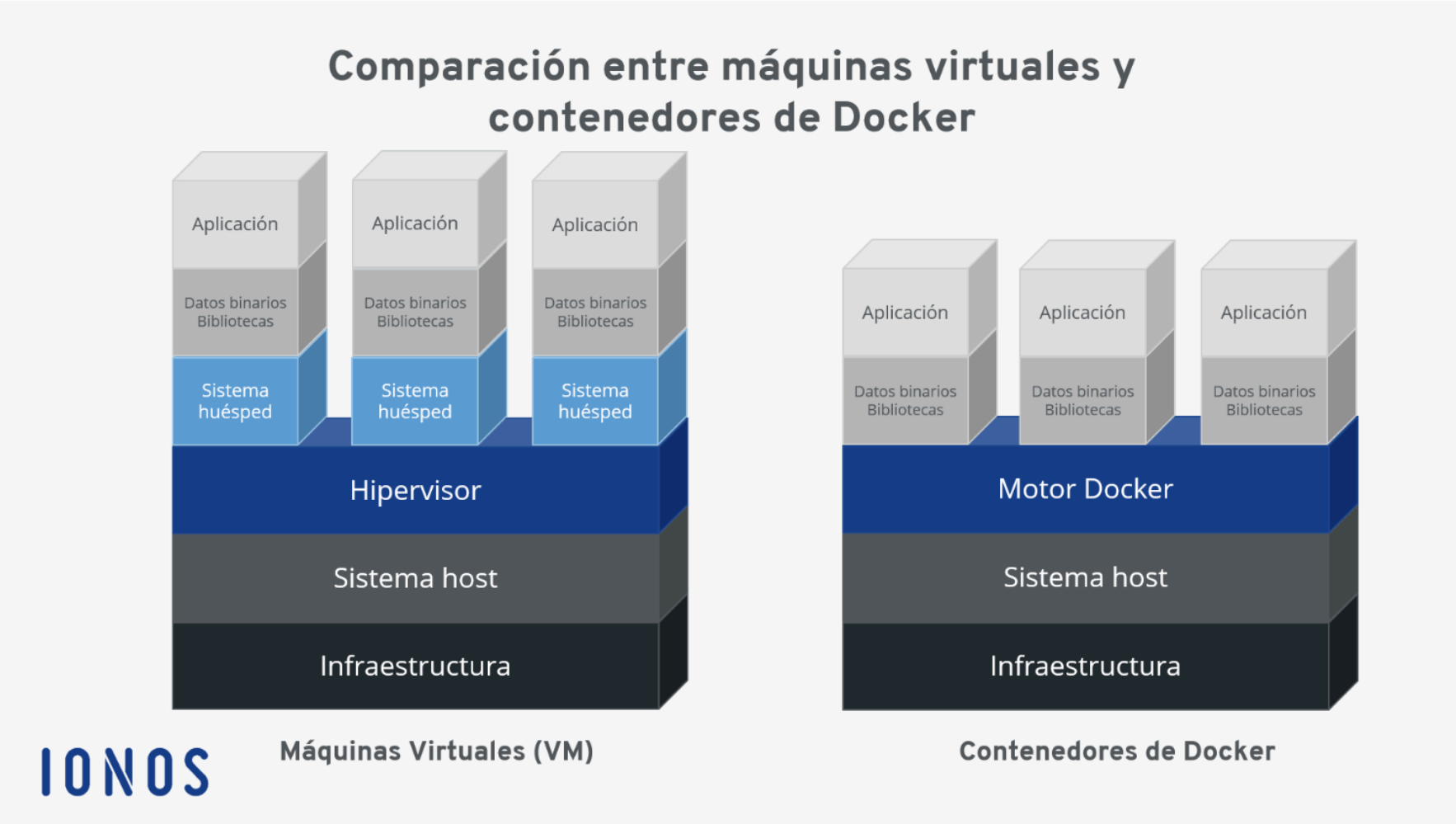

MONOLITHIC APPLICACTURE
MONOLITHIC APPLICATION
User Interfance (UI)
Ruilsiauass
Business Logic
Analytics
Reporting & Analytics
Analytics
USER
SERVER
Single Database
MICROSERVICES ARCHITECTURE
Service A (Users)
Service B (Products)
Dateless
Service C (Products)
Cervir e C (Doducts)
USER
API GATEWAY

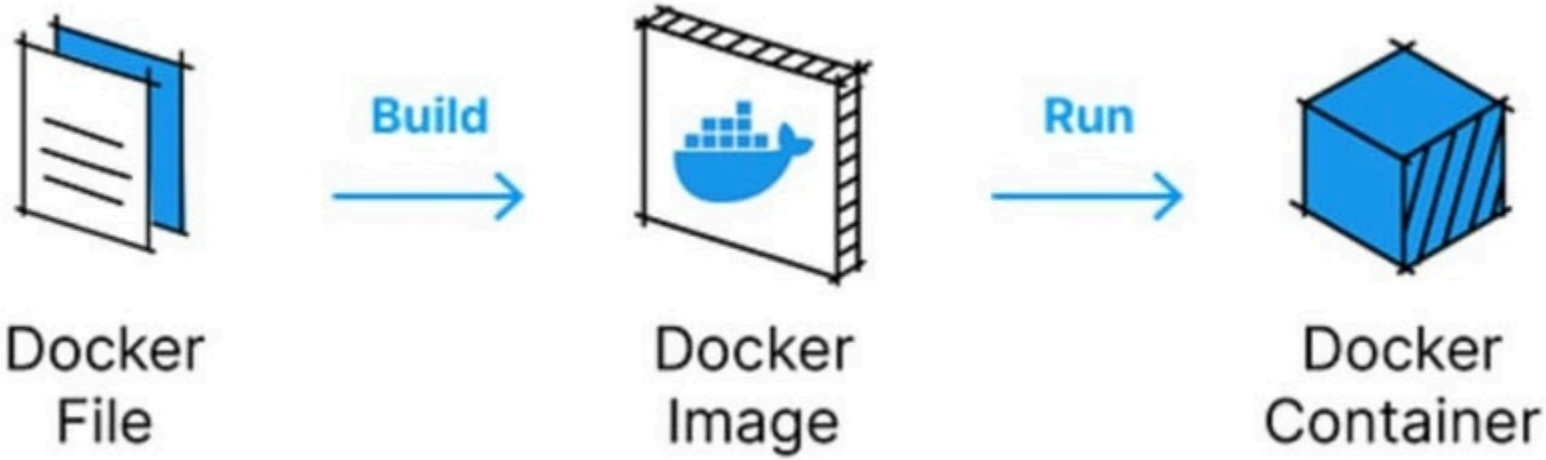
Arquitectura monolítica vs multiservicios

Aspecto	Monolítico (Sin Contenedores)	Microservicios con contenedores Docker
Despliegue	MÁS RIESGOS Y MÁS LENTO	ÁGIL Y FIABLE
	Hay que reconstruir y desplegar el sistema completo ante cualquier cambio. Alto riesgo de impacto general.	Solo se construye y despliega el contenedor del servicio modificado. Despliegue más fácil.
Escalabilidad	INEFICIENTE	EFECTIVO
	Se debe escalar toda la aplicación (levantar múltiples copias del monolito), desperdiciando recursos en módulos de baja carga.	Solo se aumenta el número de contenedores del servicio específico que necesita más capacidad.
Tecnología	RÍGIDA	FLEXIBLE
	Limitado a un único stack tecnológico y a la versión de librerías más antigua requerida por algún módulo.	Cada microservicio puede usar el mejor lenguaje o *framework* para su tarea

Virtualización vs Contenerización

Aspecto	Máquina Virtual (VM)	Contenedor (Docker)
Componente Adicional	Requiere Hypervisor (Capa intermedia de virtualización de hardware).	**No requiere Hypervisor.**
Sistema Operativo	**Cada instancia requiere un Sistema Operativo completo.**	**Comparte el Kernel del Host** (Sistema Operativo Anfitrión).
Consumo de Recursos	**Alto consumo de RAM y almacenamiento.**	**Mínimo consumo de recursos y baja sobrecarga.**
Tiempo de Arranque	**Elevado (minutos, debido al arranque completo del OS).**	**Extremadamente rápido (milisegundos, solo se inicia el proceso).**
Aislamiento	Aislamiento de hardware completo	Aislamiento de procesos a nivel de espacio de usuario.
Portabilidad	La imagen es grande (gigabytes).	La imagen es pequeña (megabytes), garantizando ejecución uniforme.


Comparación entre máquinas virtuales y contenedores de Docker
Aplicación
Datos binarios Bibliotecas
Sistema huésped
Hipervisor
Sistema host
Infraestructura
Máquinas Virtuales (VM)
Motor Docker
Contenedores de Docker
IONOS


Build
Run
Docker File
Docker Image
Docker Container

Instalación en Linux

Instalación en Entorno Debian/Ubuntu

```
# Actualización de índices e instalación de dependencias
sudo apt update
sudo apt install docker.io docker-compose -y

# Verificación del servicio
docker --version
systemctl status docker

#para la ejecución sin privilegios de superusuario (sudo):sudo usermod -aG docker $USER

newgrp docker
```

Docker Desktop en Windows

En entornos de desarrollo Windows, se utiliza **Docker Desktop**, una aplicación que integra todas las herramientas necesarias.

Características y Funciones

1. **Integración WSL 2:** Ejecuta el daemon de Docker dentro del subsistema Linux de Windows para obtener rendimiento nativo.
2. **Dashboard Gráfico (GUI):** Interfaz visual para gestionar contenedores, ver logs, inspeccionar volúmenes y gestionar imágenes sin usar la terminal.

Docker CLI. Comandos de contenedores

docker run, proceso PID 1, mantener el contenedor en running	
-d (background) crea un servidor en 2ª plano de nombre srv-web	docker run -d --name srv-web nginx
Crea un contenedor Ubuntu y asocia una terminal interactiva, El contenedor permanece activo mientras interactúas con el shell. Si no se indica comando al final del docker run se ejecuta el comando de inicio por defecto	docker run -it ubuntu
El contenedor se crea realiza ping a google y al terminar su tarea (PID 1) se elimina, (sin la opción -rm se detiene, habría que borrarlo a mano)	docker run --rm alpine ping -c 3 google.com
El servidor Nginx es un proceso de larga duración. El flag -d es suficiente para lanzarlo en detached. Los flags -it son innecesarios. El contenedor se mantiene en running Con docker exec podremos abrir una terminal interactiva -it	docker run -d nginx docker exec <ID> -it bash
El proceso PID 1 es `ls`. Este comando se ejecuta, muestra el contenido, y termina inmediatamente. Al finalizar PID 1, el contenedor se detiene.	docker run -d ubuntu ls
Inicia un contenedor basado en la imagen Nginx en modo -d detach (background). El servidor Nginx se convierte en PID 1. El contenedor permanece en running *-p 8000:80* Se asocia el puerto 8000 del host al 80 del contenedor	docker run -d --name web -p 8000:80 nginx *docker exec web service nginx reload (reinicia)*

Ejecuta un servidor Apache en Foreground, en la terminal veremos los Log del servidor, entraremos en la web con el puerto 8001 del host, si cerramos la terminal el servidor se detiene	docker run -p 8001:80 httpd
Esta es la única forma de dejar un shell activo en segundo plano. -it evita que Bash termine; -d lo manda al fondo. Los tres flags son necesarios. para que puedas usar docker exec más tarde y abrir un terminal	docker run -dit --name ubu1 ubuntu docker ps docker exec -it ubu1 bash exit docker ps
Las banderas -it fuerzan al shell bash a creer que tiene una terminal adjunta, lo que le impide terminar, **manteniendo el contenedor activo.**	docker run -dit ubuntu bash
Se inicia Nginx en segundo plano, y *la opción `-it` se ignora o es irrelevante* ya que el proceso PID 1 es un servicio de larga duración por defecto.	docker run -dit nginx
Ejecutar un comando en un contenedor	docker exec <contenedor> <comando> docker exec contenedor ip a docker exec contenedor /bin/bash
Parar contenedor	docker stop srv-web
RESUMEN Para mantener un servidor en ejecución, debemos utilizar -d y podemos ejecutar los comandos desde el host con docker exec, podemos abrir una shell /bin/bash , ejecutar comandos dentro del contenedor y al salir no se parará.	docker run -d –name apache2 httpd docker ps docker exec apache2 pwd docker exec -it apache2 /bin/bash pwd exit docker ps

Si fuerala imagen ubuntu, necesito un proceso abierto , ya que no es un servidor utilizo -dit que abre una shell interactiva en segundo plano	
start, stop, listar, borrar	
Reactivación de un contenedor existente.	docker start srv-web
listar contenedores en ejecución	docker ps
Listar todos los contenedores	docker ps -a
Listar todos los contenedores y mostrar tamaño (size)	docker ps -a -s
Listado con ID de todos los contenedores, en ejecución y parados	docker ps -aq
Borrar un contenedor	docker rm debian1
Eliminación forzosa de un contenedor	docker rm -f srv-web
Borra todos los contenedores	docker rm $(docker ps -aq)
Parar todos los contenedores	docker stop $(docker ps -a -q)
Borrar un contenedor cuando se pare (-rm)	docker run -rm -dit –name debian2 debian
Reiniciar automaticamente un contenedor *--restart unless-stopped (salvo parado manualmente)*	docker run -dit –restart always –name debian2 debian
Borrar un contenedor en modo forzoso	docker kill contenedor docker rm -f contenedor

Listar los ID de todos los contenedores	docker ps -a -q
Borrar todos los contenedores	docker rm $(docker ps -a -q)
Parar todos los contenedores	docker stop $(docker ps -a -q)
inspeccionar, LOGS, recursos en tiempo real (stats)	
Visualizar los LOGS del contenedor	docker logs contenedor
Visualizar LOGS con timestamps	docker logs -t contenedor
Visualizar LOGS en tiempo real (-f)	docker logs -tf contenedor
Visualizar procesos del contenedor	docker top contenedor
Inspeccionar un contenedor	docker inspect [ID] docker inspect [ID] | grep IP
Recursos utilizados por el contenedor	docker stats contenedor
varios	
Crear una imagen de un contenedor (se recomienda con dockerfile, no con este comando)	docker commit -m comentario IDcontenedor usuario/imagen:version
Borrar todos los contenedores parados	docker container prune

Borrar contenedores parados, redes no usadas, imágenes	docker system prune
Copiar un archivo del host al contenedor	docker cp index.html apache:/usr/local/apache2/htdocs/
copiar en carpeta de inicio de nginx Copiar la carpeta/home del contenedor en la carpeta actual del host	docker cp index.html contenedor:/usr/share/nginx/html/ docker cp contenedor:/home .
Detener de forma inmediata un contenedor	docker kill contenedor
Renombrar un contenedor	docker rename <container> <new container>

Docker CLI. Volúmenes. Persistencia de Datos

Los contenedores son inmutables y efímeros. La persistencia requiere desacoplar los datos del ciclo de vida del contenedor.

Tipos de Montaje (`-v`)

1. **Bind Mount (Desarrollo):**
   - Vinculación directa de un directorio del host al contenedor.
   - Sintaxis: `-v /ruta/host:/ruta/contenedor`
   - *Uso:* Edición de código fuente en tiempo real.

2. **Named Volume (Producción):**
   - Gestión delegada a Docker en `/var/lib/docker/volumes`.
   - Sintaxis: `-v nombre-volumen:/ruta/contenedor`
   - *Uso:* Almacenamiento de bases de datos y datos críticos.

Referencia Rápida: Servicios Comunes y Rutas (CLI)

Resumen de rutas de montaje (carpetas de inicio) y comandos de acceso rápido para los servicios más habituales.

1. Servidores Web (Volúmenes Estáticos)

Nota: Rutas internas por defecto donde el servidor busca el `index.html`.

NGINX: Ruta interna: `/usr/share/nginx/html/`

```
docker run -d -p 80:80 -v $(pwd):/usr/share/nginx/html/ nginx:alpine
```

APACHE (httpd): Ruta interna: `/usr/local/apache2/htdocs/`

```
docker run -d -p 8080:80 -v $(pwd):/usr/local/apache2/htdocs/ httpd:alpine
```

2. Base de Datos (MariaDB/MySQL)

Creación rápida y comando para entrar a la consola SQL directamente.

Crear Contenedor:

```
docker run -d --name mi-mariadb -e MYSQL_ROOT_PASSWORD=root mariadb:latest
```

Entrar a la consola SQL:

```
docker exec -it mi-mariadb mysql -u root -p
```

(El sistema pedirá la contraseña 'root' definida arriba)

Docker CLI: Diferencias Linux vs Windows

El entorno del host determina la sintaxis para rutas de volúmenes y permisos, causando errores comunes en entornos mixtos.

1. Sintaxis de Rutas (Bind Mounts)

La referencia al **directorio actua**l varía según la terminal utilizada:

- **Linux / macOS / WSL2 (Bash):** Uso de $(pwd). docker run -v $(pwd)/web:/var/www/html nginx
- **Windows (PowerShell):** Uso de ${PWD} (con llaves). docker run -v ${PWD}/web:/var/www/html nginx
- **Windows (CMD):** Uso de %cd%. docker run -v %cd%/web:/var/www/html nginx

2. Permisos

En Linux nativo se requiere sudo o grupo docker. En Docker Desktop (Windows), el usuario actual tiene permisos por defecto.

Taller CLI 1: Servidor Nginx y Apache 2

Opciones básicas de `docker run`

1. **`-d` (Detached):** Ejecución del proceso en segundo plano. Esencial para servicios persistentes.
2. **`--name`:** Asignación de identificador DNS interno y nombre de gestión.
3. **`-p host:contenedor`:** Mapeo de puertos (NAT). Permite el acceso externo al servicio.
   - Ejemplo: `-p 8080:80` redirige tráfico del puerto 8080 del host al 80 del contenedor.
4. -v **Volumen:** una carpeta del host a una carpeta del contenedor (en read only) solo se cambia la web en host
4. **`--restart always`:** Política de reinicio automático ante fallos, garantiza alta disponibilidad.
5. ***`-d`*** *(Detached, background)*

Sintaxis Completa

```
docker run -d \
--name mi-nginx-server \
-p 8080:80 \
-v $(pwd)/web:/usr/share/nginx/html:ro \
--restart always \
nginx
```

Sintaxis en Apache 2

```
docker run -d \
--name mi-apache-server \
-p 8081:80 \
-v $(pwd)/web:/usr/local/apache2/htdocs:ro \
--restart always \
```

`httpd`

Taller CLI 2: Gestión Visual con Portainer

Objetivo: Desplegar Portainer, una interfaz gráfica web para gestionar Docker, permitiendo visualizar contenedores, redes y volúmenes de forma intuitiva.

Procedimiento

1. **Creación de volumen para persistencia:** `docker volume create portainer_data`

Despliegue del contenedor: Se requiere acceso al socket de Docker (`/var/run/docker.sock`) para que Portainer pueda controlar el daemon.

```
docker run -d \
  -p 8000:8000 -p 9443:9443 \
  --name portainer \
  --restart=always \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v portainer_data:/data \
  portainer/portainer-ce:latest
```

2. **Acceso:** Abrir navegador en **`https://localhost:9443`**. Crear usuario administrador y seleccionar entorno "Local", (Portainer utiliza el puerto 8000 para conexiones a contenedores remotos)

Taller CLI 3: Despliegue de apache sobre alpine

Objetivo: Desplegar un servicio web estático accesible desde el host.

Preparación del contenido:

```
mkdir web-content
echo "<h1>Servicio Web Desplegado con Docker</h1>" > web-content/index.html
```

Ejecución del contenedor (Apache httpd):

```
docker run -d \
  --name apache-server \
  -p 8080:80 \
  -v $(pwd)/web-content:/usr/local/apache2/htdocs \
 --restart always \
  httpd:alpine
```

Validación:

- `Acceso mediante navegador: http://localhost:8080`
- `Verificación de logs: docker logs apache-server`

Borrado: `docker rm -f apache-server`

```
docker run -dit --name server-nginx -p 8081:80 nginx bash    (-dit si quiero abrir un terminal interactivo
con docker exec, el servicio no se para aunque cierres la terminal)
docker exec -it <ID> bash
```

Taller CLI 4: Despliegue de Base de Datos (MariaDB). -e variable de entorno

Objetivo: Instanciar un motor de base de datos con persistencia y variables de entorno, MariaDB y MySQL necesitan password de root para iniciarse

Ejecución del servicio (se creará el volumen mariadb_data)

```
docker run -d \
  --name db-mariadb \
  -e MARIADB_ROOT_PASSWORD=root_secret \
  -e MARIADB_DATABASE=app_db \
  -v mariadb_data:/var/lib/mysql \
  mariadb:latest
```

Interacción con la BBDD:

```
# Ejecución de cliente mysql dentro del contenedor
docker exec -it db-mariadb mariadb -u root -p
```

```
# (Ingresar contraseña: root_secret)
SQL> SHOW DATABASES;
SQL> USE app_db;
SQL> CREATE TABLE tableasir (
    id INT,
    nombre VARCHAR(50)
);
SQL> SHOW TABLES;
SQL> exit
```

Borramos el contenedor

```
docker stop <contenedor>
docker rm <contenedor>
```

Creamos un nuevo contenedor con el mismo volumen y comprobamos que persiste la tabla tableasir

```
docker run -d \
  --name maria \
  -e MARIADB_ROOT_PASSWORD=root_secret \
  -e MARIADB_DATABASE=app_db \
  -v mariadb_data:/var/lib/mysql \
  mariadb:latest

docker exec -it maria mariadb -u root -p

SQL> SHOW DATABASES;
SQL> USE app_db;
SQL> SHOW TABLES;
```

```
SQL> exit
```

```
MariaDB [app_db]> show tables
    -> ;
+------------------+
| Tables_in_app_db |
+------------------+
| tableasir        |
+------------------+
1 row in set (0.001 sec)
```

Taller CLI 5: Interconexión Manual Web-SQL (Bridge)

Objetivo: Despliegue manual de una arquitectura cliente-servidor aislada en una red tipo Bridge con persistencia de datos.

1. Infraestructura de Red y Datos

Creación del segmento de red aislado y volúmenes para persistencia.

```
docker network create blogred
docker volume create blogwebdata
```

```
docker volume create blogdbdata
```

2. Despliegue de Base de Datos (Backend)

Ejecución del motor MySQL conectado a la red `blogred`.

```
docker run -d \
  --name db \
  --network blogred \
  -e MYSQL_ROOT_PASSWORD=pw123 \
  -e MYSQL_DATABASE=dbdata \
  -v blogdbdata:/var/lib/mysql \
  mysql
```

3. Despliegue de Servidor Web (Frontend)

Ejecución de Apache con PHP, exponiendo el puerto 80.

```
docker run -d \
  --name web \
  --network blogred \
  -p 80:80 \
  -v blogwebdata:/var/www/html \
  php:apache
```

4. Validación de Conectividad

Instalación de cliente MySQL en el servidor web para verificar acceso al backend.

```
docker exec -it web bash

# Dentro del contenedor:
apt update && apt install -y default-mysql-client
mysql -h db -u root -p
# (Password: pw123)
# SQL> show databases;
exit;
```

Taller CLI 6. Desplegar 3 servidores nginx:alpine

Realizar el despliegue con 3 webs elaboradas propias o descargadas de alguna web de plantillas gratuitas *www.plantillashtmlgratis.com*

Creo una carpeta del proyecto y descargo 3 plantillas WEB HTML de cualquier página web

```
wget https://plantillashtmlgratis.com/wp-content/themes/helium-child/descargas/page273/speed.zip
wget https://plantillashtmlgratis.com/wp-content/themes/helium-child/descargas/page281/cs.zip
wget https://plantillashtmlgratis.com/wp-content/themes/helium-child/descargas/page256/central.zip

unzip speed.zip
unzip cs.zip
unzip central.zip
```

Desplegar los 3 contenedores en los puertos 8081, 8082 y 8083 del host

```
 docker run -d --name web1 -p 8081:80 -v ./cs:/usr/share/nginx/html:ro nginx:alpine
 docker run -d --name web2 -p 8082:80 -v ./speed:/usr/share/nginx/html:ro nginx:alpine
 docker run -d --name web3 -p 8083:80 -v ./2103_central:/usr/share/nginx/html:ro nginx:alpine
```

`docker ps -s` # Compruebo los 3 contenedores en ejecución

`ss -tlpan` #Compruebo puertos abiertos en el host

Accedo desde el navegador a los 3 servidores web
http://localhost:8081

http://localhost:8082

http://localhost:8083

En Windows:

```
D:\docker>docker run -dit --name web3 -p 8083:80 -v %cd%\web3:/usr/share/nginx/html:ro nginx:alpine
da3f521d8d42b7ed34869e244344779c7898b1364b24ac02b30bfd2c6fe1109d

D:\docker>docker run -dit --name web2 -p 8082:80 -v %cd%\web2:/usr/share/nginx/html:ro nginx:alpine
eb00e720c2a344687c6c5dfc86e47c4ad67df6d0e88692a0f386052fa5955d02

D:\docker>docker run -dit --name web1 -p 8081:80 -v %cd%\web1:/usr/share/nginx/html:ro nginx:alpine
7fb22c0c04fe98f277879a65ac84e8a957e795fe542ec95128f0f4536b9d43c0

D:\docker>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS                   NAMES
7fb22c0c04fe   nginx:alpine   "/docker-entrypoint.…"   5 seconds ago    Up 3 seconds    0.0.0.0:8081->80/tcp    web1
eb00e720c2a3   nginx:alpine   "/docker-entrypoint.…"   14 seconds ago   Up 13 seconds   0.0.0.0:8082->80/tcp    web2
da3f521d8d42   nginx:alpine   "/docker-entrypoint.…"   25 seconds ago   Up 23 seconds   0.0.0.0:8083->80/tcp    web3
```

Realizar el despliegue con 3 webs elaboradas propias o descargadas de alguna web de plantillas gratuitas
www.plantillashtmlgratis.com

Taller CLI 7. Desplegar Nextcloud. Nube privada

Ejemplo con volúmenes

$ docker volume create nextcloud creo un 'contenedor volumen'

Y creamos el contenedor nextcloud, guardando el directorio /var/www/html del contenedor en el volumen creado:

$ docker run -d -p 8081:80 -v nextcloud:/var/www/html --name contenedor_nextcloud nextcloud

Comprobamos que podemos acceder, terminamos de configurar la aplicación y una vez operativa subimos un fichero a la aplicación nextcloud.

A continuación eliminamos el contenedor y creamos uno nuevo con el mismo volumen:

$ docker rm -f contenedor_nextcloud

```
$ docker run -d -p 80:80  -v nextcloud:/var/www/html --name contenedor_nextcloud nextcloud
```

Accede de nuevo a la aplicación nextcloud comprueba que la aplicación sigue configurada y que los ficheros subidos no se han perdido.

Ejemplo con bind mount

En este caso, vamos a crear un directorio en nuestro ordenador, que es el que vamos a montar en el contenedor:

mkdir datos_nextcloud
Y creamos el contenedor con la siguiente instrucción:

```
docker run -d -p 80:80 -v
/Users/juancarlossanroman/docker-ejemplos/datos_nextcloud:/var/www/html  --name
contenedor_nextcloud nextcloud
```

Volvemos a acceder, configuramos la aplicación y subimos algún fichero. Usando bind mount tenemos acceso al directorio:

$ cd datos_nextcloud/

~/datos_nextcloud$ ls

Podemos comprobar que al eliminar el contenedor y crearlo de nuevo usando el mismo directorio bind mount, toda la configuración y los ficheros subidos no se han perdido.

Taller CLI 8. Desplegar la aplicación Temperaturas

Vamos a hacer un despliegue completo de una aplicación llamada Temperaturas. Esta aplicación nos permite consultar la temperatura mínima y máxima de todos los municipios de España. Esta aplicación está formada por dos microservicios:

- frontend: Es una aplicación escrita en Python que nos ofrece una página web para hacer las búsquedas y visualizar los resultados. Este microservicio hará peticiones HTTP al segundo microservicio para obtener la información. Este microservicio ofrece el servicio en el puerto 3000/tcp. Usaremos la imagen iesgn/temperaturas_frontend.
- backend: Es el segundo microservicio que nos ofrece un servicio web de tipo API Restful. A esta API Web podemos hacerles consultas sobre los municipios y sobre las temperaturas. En este caso, se utiliza el puerto 5000/tcp para ofrecer el servicio. Usaremos la imagen iesgn/temperaturas_backend.

El microservicio frontend se conecta a backend usando el nombre temperaturas-backend. Por lo tanto el contenedor con el microservicio backend tendrá ese nombre para disponer de una resolución de nombres adecuada en el dns.

Vamos a crear una red para conectar los dos contenedores:

```
$ docker network create red_temperaturas
```

Para ejecutar los contenedores:

```
$ docker run -d --name temperaturas-backend --network red_temperaturas iesgn/temperaturas_backend
```

```
$ docker run -d -p 80:3000 --name temperaturas-frontend --network red_temperaturas iesgn/temperaturas_frontend
```

Algunas observaciones:

- Este es un tipo de aplicación, que se caracteriza por no necesitar guardar información para su funcionamiento. Son las denominadas **aplicaciones sin estado**.
- No es necesario mapear el puerto de backend, ya que no vamos a acceder desde el exterior. Sin embargo el microservicio frontend va a poder acceder a backend al puerto 5000 porque están conectado a la misma red.
- Al nombrar al contenedor de la base de datos con temperaturas-backend se crea una entrada en el DNS que resuelve ese nombre con la ip del contenedor. Como hemos indicado, por defecto, el microservicio frontend usa ese nombre para acceder.

Fuente:
https://iesgn.github.io/curso_docker_2021/sesion4/temperaturas.html

Anexo. Docker -v. Volúmenes bind

- CMD (Windows):
    - Ejemplo: -v C:\data_host:/app/data
    - Nota: Usa \ y requiere la letra de unidad (C:).
- PowerShell:
    - Ejemplo: -v C:/data_host:/app/data
    - Nota: Usa / y requiere la letra de unidad (C:).
- WSL/Bash (Accediendo a Windows):
    - Ejemplo: -v /mnt/c/data_host:/app/data
    - Nota: Usa /mnt/c para acceder a la unidad de Windows.
- WSL/Bash (Ruta NATIVA de WSL):
    - Ejemplo: -v /home/usuario/folder:/app/data
    - Nota: Usa la ruta directa de Linux si la carpeta está dentro de WSL.
- Linux (Host Nativo):
    - Ejemplo: -v /home/user/data:/app/data

- Nota: Formato estándar de Linux.

Taller Compose 1: Despliegue Stack LAMP (WordPress)

```
version: '3.8'
services:
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: admin_pass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wp_user
      MYSQL_PASSWORD: wp_pass
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - wp_net
  wordpress:
    image: wordpress:latest
    restart: unless-stopped
    ports:
      - "80:80"
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_USER: wp_user
      WORDPRESS_DB_PASSWORD: wp_pass
      WORDPRESS_DB_NAME: wordpress
    depends_on:
      - db
    networks:
      - wp_net
volumes:
  db_data:
networks:
  wp_net:
```

Ejecución: `docker-compose up -d`

Taller Compose 2: Despliegue Nextcloud

```
version: '3.8'
services:
  nextcloud-db:
    image: mariadb:10.6
    restart: always
    command: --transaction-isolation=READ-COMMITTED --binlog-format=ROW
    volumes:
      - db_nc:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: root_pass
      MYSQL_PASSWORD: nc_pass
      MYSQL_DATABASE: nextcloud
      MYSQL_USER: nextcloud
  app:
    image: nextcloud
    restart: always
    ports:
      - "8080:80"
    volumes:
      - nextcloud_data:/var/www/html
    environment:
      MYSQL_HOST: nextcloud-db
      MYSQL_PASSWORD: nc_pass
      MYSQL_DATABASE: nextcloud
      MYSQL_USER: nextcloud
    depends_on:
      - nextcloud-db
volumes:
  db_nc:
  nextcloud_data:
```

Ejecución: `docker-compose up -d`

Validación: Acceso a `http://localhost:8080`

Taller Compose 3: Maria DB

Archivo `docker-compose.yml`:

```
services:
  db:
    container_name: contenedor_mariadb
    image: mariadb
    restart: always
    environment:
      MARIADB_ROOT_PASSWORD: sanluis
    volumes:
      - ./datos:/var/lib/mysql
```

Ejecución: `docker-compose up -d`

Ejemplo adminer controlador de la base de datos mariadb

```
# Use root/example as user/password credentials
services:
  db:
    image: mariadb
    restart: always
    environment:
      MARIADB_ROOT_PASSWORD: example
  adminer:
    image: adminer
    restart: always
    ports:

      - 8080:8080
```

Taller Compose 4: MySQL . variables en archivo .env

Archivo .env (Definición de variables)

```
# .env
# Versión de MySQL (puedes usar 5.7 o 8.0)
MYSQL_VERSION=8.0

# Configuración de Puertos
HOST_PORT=3306

# Credenciales de Superusuario (Obligatorio)
MYSQL_ROOT_PASSWORD=root_secreto_123

# Credenciales para tu Aplicación
DBMYSQL_DATABASE=mi_aplicacion_db
MYSQL_USER=usuario_app
MYSQL_PASSWORD=usuario_secreto_123
```

Archivo docker-compose.yaml

```
version: '3.8'
services:
  mysqldb:
    image: mysql:${MYSQL_VERSION}
    container_name: mysql_container
    restart: always

    # Mapeo de variables de entorno específicas de MySQL
    environment:
      MYSQL_ROOT_PASSWORD:       ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}

    # Mapeo de puertos: Host:Contenedor
    ports:
      - "${HOST_PORT}:3306"
    # Persistencia de datos
    volumes:
      - mysql_data:/var/lib/mysql
volumes:
  mysql_data:
```

Docker-compose comprueba si en la misma carpeta existe el archivo .env en cuyo caso crea esas variables

Docker-compose.yaml

```
services:
  mysql:
    image: mysql:8.0 # Versión de MySQL, puedes cambiarla
    container_name: mysql_server
    restart: always
    environment:
      # Variables de entorno de uso común para MySQL
      # Se leen directamente desde el archivo .env
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}

      # Opcional: Define la codificación de caracteres predeterminada
      # LANG: C.UTF-8

    ports:
      # Mapeo de puertos: HOST_PORT:CONTAINER_PORT
      - "${DB_PORT}:3306"

    volumes:
      # Mantiene los datos persistentes en el volumen llamado 'mysql_data'
      - mysql_data:/var/lib/mysql

# Definición de volúmenes
```

```
volumes:
  mysql_data:
```

Cómo ejecutarlo y probarlo:

Levanta el servicio: `docker-compose up -d`

Verifica que está corriendo: `docker-compose ps`

Prueba de conexión (desde la terminal): `docker exec -it mysql_container mysql -u usuario_app -p`

Taller docker-compose 5. Calculadora

Crear en carpeta calculadora/docker-compose.yaml

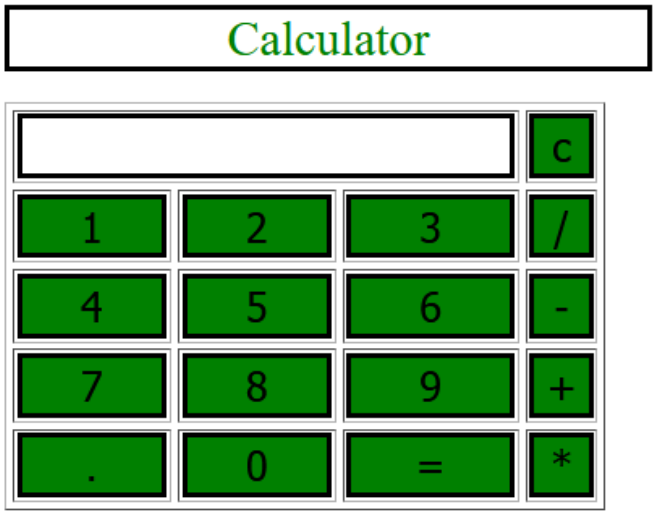
```
services:
  calculadora:
   image: redxlus/just-template-http:calculadora
   ports:
    - "80:80"
   restart: always
```

```
  GNU nano 6.2                    docker-compose.yaml
services:
  calculadora:
   image: redxlus/just-template-http:calculadora
   ports:
    - "80:80"
   restart: always
```

```
docker-compose up -d            #Levantar servicios

http:localhost          # Acceder al servicio puerto
80

docker-compose down     # Eliminar los servicios
```



Taller docker-compose 6. Repositorio awesome-compose

En github hay infinididad de repositorios con herramientas interesantes, busca entre los repositorios de github, selecciona alguno, lee los README y sigue las instrucciones.

Por ejemplo PLEX o PROMETHEUS-GRAFANA.

https://github.com/docker/awesome-compose.git